Eleeveon Schools Advanced Timetable and Scheduling Engine

Complete Master Implementation Plan

1. Final vision

The timetable module will evolve from a manual timetable-recording system into a complete scheduling environment that can:

- Define school days and period structures.
- Reserve assembly, worship, breaks, lunch, sports, clubs, and closing.
- Store teacher availability and workload limits.
- Define how many periods every class and subject needs.
- Schedule single classes.
- Schedule two or more classes together.
- Schedule one or multiple teachers together.
- Assign rooms, laboratories, equipment, and other resources.
- Validate manual timetable changes immediately.
- Generate several timetable drafts automatically.
- Score and compare generated drafts.
- Explain why some requirements could not be placed.
- Recommend reasonable class combinations when the timetable is extremely constrained.
- Apply recommendations only after administrator approval.
- Regenerate only affected or unlocked sessions.
- Publish one official timetable.
- Produce class, teacher, student, parent, room, and branch timetable views.
- Print professional weekly timetables.

The final workflow will be:

```
School structure and period setup
              ↓
Teaching requirements
              ↓
Availability and constraints
              ↓
Readiness validation
              ↓
Standard draft generation
              ↓
Draft optimisation
              ↓
```

```
Recovery analysis when necessary
              ↓
Administrator-approved structural changes
              ↓
Regeneration
              ↓
Manual review and locking
              ↓
Commit and publish
              ↓
Role-based timetable projections
```

---

2. Core architectural decisions

2.1 One timetable, multiple projections

Eleeveon will maintain one authoritative branch timetable.

It will not separately create unrelated class, teacher, student, and room timetables.

```
Published Branch Timetable
├── Class timetable
├── Teacher timetable
├── Student timetable
├── Parent-child timetable
├── Room timetable
├── Resource timetable
└── Printable timetable
```

The same session record will appear wherever it is relevant.

2.2 Sessions may have multiple classes

A timetable session may apply to:

- One class.
- Multiple classes.
- An academic level.
- A programme.
- A curriculum pathway.
- A saved custom group.

- Selected students.
- The entire branch.

The current `ScheduleSession` contains optional `classId` and `teacherId` references. These can remain as primary compatibility references, but they will no longer represent the full list of participants.

Authoritative participation will use relational tables.

2.3 Sessions may have multiple teachers

Examples include:

- Lead teacher and assistant teacher.
- Two teachers supervising physical education.
- Several teachers supervising assembly.
- Team teaching.
- Club facilitators.
- Practical sessions requiring a teacher and laboratory assistant.

A primary `teacherId` can remain on the session for compatibility, while a session-teacher table stores every participating teacher.

2.4 Combined and separate delivery are different

Suppose JHS 1A and JHS 1B each need two Computing periods.

Separate delivery

```
JHS 1A: 2 sessions
JHS 1B: 2 sessions
Total: 4 sessions
```

Combined delivery

```
JHS 1A + JHS 1B: 2 shared sessions
Total: 2 sessions
```

Every requirement involving multiple classes must therefore store:

```
deliveryGrouping: "separate" | "combined";
```

2.5 Permanent combinations and suggested combinations are different

The system will support:

Permanent combination

The school deliberately teaches the classes together.

Example:

```
JHS 1A + JHS 1B Computing
```

Subject-specific combination

The classes combine only for one subject.

Example:

```
Primary 5A + Primary 5B French
```

Partial combination

Some periods are combined while others remain separate.

Example:

```
Computing:
1 combined demonstration period
2 separate practical periods
```

Recovery combination

The generator proposes combining compatible classes because the original timetable cannot fit.

A recovery combination is never applied or published automatically.

2.6 Hard constraints and soft constraints

Hard constraints

A timetable placement is invalid when it causes:

- Teacher collision.
- Class collision.
- Student-group collision.
- Room collision.
- Resource collision.
- Teacher unavailable.
- Multiple required teachers not simultaneously available.
- Teaching during a blocked activity.
- Invalid period duration.
- Incompatible class period templates.
- Room capacity failure.
- Required equipment unavailable.
- Maximum hard workload exceeded.
- Required consecutive periods unavailable.

Soft constraints

A placement may be allowed but receive a penalty when it:

- Uses an avoided teacher period.
- Gives a teacher too many gaps.
- Schedules difficult subjects late.
- Places the same subject repeatedly on one day.
- Produces an uneven class workload.
- Uses a non-preferred room.
- Breaks a preferred free day.
- Produces too many consecutive periods.
- Gives poor subject distribution.
- Uses a less desirable class combination.

---

3. Final implementation phases

The project will be completed in **ten major phases**.

```
Phase 1 — Database, migration and sync foundation
Phase 2 — Scheduling domain and participant resolution
Phase 3 — Period templates and shared blocks
Phase 4 — Traditional timetable grid and manual planning
```

```
Phase 5 — Teacher availability and workload
Phase 6 — Subject requirements, groups and resources
Phase 7 — Constraint, conflict and readiness engines
Phase 8 — Standard timetable generation and draft scoring
Phase 9 — Recovery analysis and class-combination recommendations
Phase 10 — Publishing, printing and portal projections
```

Phase 1 — Database, migration and sync foundation

Goal

Extend the current scheduling schema so it can store:

- Period definitions.
- Constraints.
- Participant groups.
- Teaching requirements.
- Drafts.
- Generation results.
- Recovery suggestions.
- Published versions.

The current Dexie class exposes only:

```
scheduleTimetables
scheduleSessions
scheduleResources
scheduleConflicts
```

as scheduling tables.

Existing frontend files to update

```
app/lib/db/db.ts
app/lib/db/db-version.ts
app/lib/db/db-migrations.ts
app/lib/constants/syncStatus.ts, only if needed
app/hooks/useDataRevision.ts
```

Existing sync files to update

Use the actual names present in the codebase, including the relevant files under:

```
app/lib/sync/
```

Likely responsibilities include:

```
sync registry
push-table registry
pull/bootstrap registry
table-name mapping
payload preparation
soft-delete mapping
local-first table registration
```

Backend files to update

```
backend/prisma/schema.prisma
backend/src/sync/
backend/src/scheduling/
```

New scheduling types

Add or extend:

```
ScheduleSlotType
ScheduleParticipantType
ScheduleGroupType
ScheduleDeliveryGrouping
ScheduleTeacherRole
ScheduleAvailabilityType
ScheduleConstraintStrength
ScheduleConstraintCategory
ScheduleDraftStatus
ScheduleGenerationStatus
SchedulePlacementSource
ScheduleIssueType
ScheduleSuggestionType
ScheduleSuggestionStatus
```

```
ScheduleCombinationPolicy
SchedulePublishStatus
```

Suggested types:

```ts
type ScheduleDeliveryGrouping =
  | "separate"
  | "combined";

type ScheduleAvailabilityType =
  | "available"
  | "unavailable"
  | "preferred"
  | "avoid";

type ScheduleConstraintStrength =
  | "hard"
  | "strong"
  | "preferred"
  | "avoid";

type ScheduleCombinationPolicy =
  | "never"
  | "suggest_only"
  | "allowed"
  | "preferred";

type SchedulePlacementSource =
  | "manual"
  | "generated"
  | "locked"
  | "shared_block"
  | "recovery_suggestion";
```

New database tables

A. Period and school-day structure

```
schedulePeriodTemplates
schedulePeriodTemplateAssignments
schedulePeriodSlots
scheduleSharedBlocks
```

```
scheduleSharedBlockGroups
scheduleSharedBlockTeachers
```

B. Saved participant groups

```
scheduleGroups
scheduleGroupMembers
```

C. Teacher constraints

```
scheduleTeacherAvailability
scheduleTeacherWorkloadRules
```

D. Subject and lesson requirements

```
scheduleSubjectRequirements
scheduleRequirementGroups
scheduleRequirementTeachers
scheduleResourceRequirements
scheduleConstraintRules
```

E. Official session relationships

```
scheduleSessionGroups
scheduleSessionTeachers
scheduleSessionResources
```

The current single `resourceId` can remain as the primary resource, while the relationship table supports several required resources.

F. Generation and draft storage

```
scheduleGenerationRuns
scheduleDrafts
scheduleDraftSessions
scheduleDraftSessionGroups
scheduleDraftSessionTeachers
scheduleDraftSessionResources
scheduleGenerationIssues
```

G. Recovery and recommendation storage

```
scheduleGenerationSuggestions
scheduleSuggestionRequirements
scheduleSuggestionGroups
scheduleSuggestionTeachers
scheduleSuggestionResources
```

H. Optional publishing and version history

```
schedulePublishEvents
scheduleVersionSnapshots
```

Extend `ScheduleTimetable`

Add:

```ts
periodTemplateId?: string | null;

publishedDraftId?: string | null;
generationRunId?: string | null;

versionNumber?: number;
publishStatus?: "draft" | "published" | "superseded";

publishedAt?: number | null;
publishedByUserId?: string | null;

supersedesTimetableId?: string | null;
```

The current timetable model already stores academic context, scope, effective dates, status, and creator information. These new fields will add generation and publishing history without removing the existing structure.

Extend `ScheduleSession`

Add:

```ts
periodSlotId?: string | null;
requirementId?: string | null;
```

```
draftId?: string | null;
sharedBlockId?: string | null;

primaryClassId?: string | null;

audienceType?:
  | "single_class"
  | "multiple_classes"
  | "academic_level"
  | "programme"
  | "pathway"
  | "branch"
  | "custom_group"
  | "selected_students";

deliveryGrouping?: "separate" | "combined";

placementSource?:
  | "manual"
  | "generated"
  | "locked"
  | "shared_block"
  | "recovery_suggestion";

locked?: boolean;
lockedReason?: string | null;
lockedByUserId?: string | null;
lockedAt?: number | null;

generationScore?: number | null;
combinationSuggestionId?: string | null;
```

Retain:

```
classId?: string | null;
teacherId?: string | null;
resourceId?: string | null;
```

as primary references for backward compatibility.

Phase 1 migration rules

The migration must:

1. Preserve all current timetable records.
2. Preserve current sessions.
3. Convert existing `classId` into one `scheduleSessionGroups` record.
4. Convert existing `teacherId` into one `scheduleSessionTeachers` record.
5. Convert existing `resourceId` into one `scheduleSessionResources` record.
6. Mark existing placements as `manual`.
7. Avoid opening a second Dexie instance.
8. Register every new table in sync.
9. Ensure rollback/recovery remains compatible with the existing migration protection system.

Phase 1 completion standard

Phase 1 is complete when:

- The new schema opens successfully.
- Existing data remains visible.
- New tables can create, update, soft-delete, push, pull, and bootstrap.
- Single-class sessions continue working.
- Multi-class sessions can be stored.
- Multi-teacher sessions can be stored.
- Drafts remain separate from official sessions.
- Recovery suggestions can be persisted.

Phase 2 — Scheduling domain and participant resolution

Goal

Create a reusable scheduling domain that is independent of React pages.

New folder

```
app/lib/scheduling/
```

New files

`types.ts`

Contains normalized in-memory engine types:

```
SchedulingInput
SchedulingPeriodTemplate
SchedulingSlot
SchedulingRequirement
SchedulingParticipant
SchedulingGroup
SchedulingTeacher
SchedulingResource
SchedulingSharedBlock
SchedulingConstraint
CandidatePlacement
PlacedSession
ConstraintViolation
GenerationIssue
GenerationSuggestion
DraftSolution
DraftScore
GenerationProgress
```

`groupResolver.ts`

Functions:

```
resolveGroupClasses()
resolveGroupStudents()
resolveGroupTeachers()
resolveGroupProgrammes()
resolveGroupPathways()

resolveSessionClassIds()
resolveSessionStudentIds()
resolveSessionTeacherIds()
resolveSessionResourceIds()

resolveRequirementClassIds()
resolveRequirementTeacherIds()
resolveRequirementResourceIds()

groupContainsClass()
```

```
groupContainsStudent()
groupsOverlap()
```

`timetableRepository.ts`

Functions:

```
loadPlanningContext()
loadTimetable()
loadPublishedTimetable()
loadDraft()
loadRequirements()
loadConstraints()
loadParticipantGroups()

saveDraft()
saveGenerationRun()
saveGenerationIssues()
saveGenerationSuggestions()

commitDraft()
publishTimetable()
archiveTimetableVersion()
```

`timetableProjection.ts`

Functions:

```
projectBranchTimetable()
projectClassTimetable()
projectTeacherTimetable()
projectStudentTimetable()
projectParentChildTimetable()
projectRoomTimetable()
projectResourceTimetable()
```

`scheduleIdentity.ts`

Functions:

```
buildSessionFingerprint()
buildRequirementFingerprint()
buildParticipantFingerprint()
```

```
buildConflictFingerprint()
buildDraftFingerprint()
```

This prevents duplicate projections and duplicate generation records.

`scheduleSelectors.ts`

Reusable selectors for:

```
selectActiveTimetable()
selectPublishedTimetable()
selectSessionsForClass()
selectSessionsForTeacher()
selectSessionsForStudent()
selectSessionsForRoom()
selectCombinedSessions()
```

New hook

```
app/hooks/useSchedulingRevision.ts
```

It listens to all scheduling-related table changes.

Phase 2 completion standard

- One combined session can appear in several class views.
- It appears only once in branch totals.
- It appears only once in each teacher's schedule.
- Student and parent projection can resolve custom group membership.
- All scheduling pages use the same participant resolution rules.

Phase 3 — Period templates and shared blocks

Goal

Allow each school to define its real operating day before lessons are scheduled.

New domain file

```
app/lib/scheduling/periodEngine.ts
```

Functions:

```
resolvePeriodTemplate()
resolveTemplateForClass()
resolveTemplateForLevel()
resolveTemplateForDay()

getSlotsForDay()
getTeachingSlots()
getBlockedSlots()
getConsecutiveSlots()

slotsOverlap()
slotDuration()
slotMatchesTimeRange()

areTemplatesCompatible()
findCommonSlotsBetweenGroups()
```

New components

```
app/components/scheduling/PeriodTemplateEditor.tsx
app/components/scheduling/PeriodSlotEditor.tsx
app/components/scheduling/PeriodTemplateAssignmentEditor.tsx
app/components/scheduling/SharedBlockEditor.tsx
app/components/scheduling/SharedBlockScopeEditor.tsx
```

Period template capabilities

Schools can define:

- Operating days.
- Day start and closing time.
- Period names.
- Period start and end.
- Teaching periods.
- Assembly.
- Worship.
- Break.

- Lunch.
- Sports.
- Club periods.
- Staff meeting.
- Closing.
- Friday-specific structures.
- Separate templates for primary, JHS, SHS, or selected classes.

Combined-class compatibility rule

For the first version:

> Classes can be combined only when their selected period slots have matching start and end times.

Later versions may support partial overlap, but not during the first implementation.

Phase 3 completion standard

- A branch can create one or several period templates.
- Different class levels can use different templates.
- Shared activities reserve periods automatically.
- The engine can find compatible common periods for combined groups.

Phase 4 — Traditional timetable grid and manual planning

Goal

Build a familiar school timetable interface using the period definitions from Phase 3.

New shared components

```
app/components/scheduling/WeeklyTimetableGrid.tsx
app/components/scheduling/TimetableCell.tsx
app/components/scheduling/PeriodHeader.tsx
app/components/scheduling/TimetableGridToolbar.tsx
app/components/scheduling/TimetableSessionEditor.tsx
app/components/scheduling/SessionParticipantSummary.tsx
app/components/scheduling/SessionLockControl.tsx
```

Grid structure

Recommended primary layout:

```
            Period 1  Period 2  Break  Period 3  Lunch  Period 4
Monday
Tuesday
Wednesday
Thursday
Friday
```

Capabilities:

- Sticky period headers.
- Sticky day labels.
- Horizontal mobile scrolling.
- Compact cards.
- Double-period spanning.
- Shared-block rendering.
- Dark mode.
- Theme colours.
- Combined-class labels.
- Multiple-teacher labels.
- Room and resource indicators.
- Locked-session indicators.
- Generated/manual indicators.
- Conflict badges.
- Print mode.

Existing files to rebuild

```
app/branch-admin/BranchTimetable.tsx
app/branch-admin/ClassTimetable.tsx
```

Manual actions

Administrators can:

- Add a lesson.
- Move a lesson.
- Swap lessons.
- Duplicate a lesson.
- Combine classes.
- Split a combined session.

- Add another teacher.
- Change room.
- Lock a session.
- Unlock a session.
- Mark a session as fixed.
- Delete or soft-delete a session.

Phase 4 completion standard

- Existing sessions appear in the new grid.
- Single and combined sessions can be created manually.
- Editing from one class updates all participating class views.
- Locked sessions are visibly protected.
- The grid works on mobile, tablet, desktop, and print.

Phase 5 — Teacher availability and workload

Goal

Turn teacher workload and availability into persistent scheduling inputs.

New domain files

```
app/lib/scheduling/availabilityEngine.ts
app/lib/scheduling/workloadEngine.ts
```

`availabilityEngine.ts`

Functions:

```
isTeacherAvailable()
getTeacherAvailabilityState()
getTeacherBlockedSlots()
getTeacherPreferredSlots()
getTeacherAvoidedSlots()
explainTeacherAvailability()
validateAvailabilityAgainstSessions()
findCommonTeacherAvailability()
```

`workloadEngine.ts`

Functions:

```
calculateDailyLoad()
calculateWeeklyLoad()
calculateConsecutivePeriods()
calculateFreePeriods()
calculateTeacherGaps()
calculateRequiredPeriods()
calculateScheduledPeriods()
calculateRemainingPeriods()

validateTeacherWorkload()
scoreTeacherWorkloadBalance()
```

Existing files to rebuild

```
app/branch-admin/modules/TeacherAvailability.tsx
app/branch-admin/modules/TeacherWorkload.tsx
app/branch-admin/modules/TeacherTimetable.tsx
```

Availability editor features

- Select teacher.
- Weekly period grid.
- Available.
- Unavailable.
- Preferred.
- Avoid.
- Drag across cells.
- Copy Monday pattern to another day.
- Apply a rule to multiple teachers.
- Add reason.
- Make a rule hard or soft.
- Show existing session violations.

Workload rule features

- Maximum periods daily.
- Maximum periods weekly.
- Maximum consecutive periods.
- Minimum free periods daily.
- Preferred free day.

- Lunch protection.
- Allow overload.
- Overload approval reason.
- Hard or soft enforcement.

Workload counting rules

A lesson involving two classes and one teacher counts as:

```
Teacher: 1 period
Class A: 1 period
Class B: 1 period
```

A lesson involving two teachers counts as one period for each teacher.

Phase 5 completion standard

- Availability is stored, not only inferred.
- Workload limits are stored.
- Existing timetables can be audited against the new rules.
- The generator can use teacher availability and workload as inputs.

Phase 6 — Subject requirements, groups and resources

Goal

Describe everything the timetable must contain before generation begins.

New domain file

```
app/lib/scheduling/requirementBuilder.ts
```

Functions:

```
buildRequirementsForTimetable()
buildRequirementsFromClassSubjects()
expandSeparateRequirements()
preserveCombinedRequirements()
buildPartialCombinationRequirements()
```

```
validateRequirementCompleteness()
findMissingTeachers()
findMissingPeriodCounts()
findMissingRooms()
findInvalidGroupCombinations()
```

New components

```
app/components/scheduling/SubjectRequirementEditor.tsx
app/components/scheduling/ParticipantGroupEditor.tsx
app/components/scheduling/ScheduleGroupManager.tsx
app/components/scheduling/ScheduleGroupMemberEditor.tsx
app/components/scheduling/RequirementTeacherEditor.tsx
app/components/scheduling/ResourceRequirementEditor.tsx
app/components/scheduling/RequirementDistributionEditor.tsx
```

Requirement fields

Each subject requirement can include:

```
subjectId
classSubjectId

periodsPerWeek
minimumPeriodsPerWeek
maximumPeriodsPerWeek
maximumPeriodsPerDay

preferredBlockSize
minimumBlockSize
maximumBlockSize
requiresDoublePeriod

deliveryGrouping
combinedPeriodsPerWeek
separatePeriodsPerWeek

combinationPolicy
maxCombinedClasses
maxCombinedStudents

preferredDays
blockedDays
```

```
preferredPeriodSlotIds
avoidedPeriodSlotIds

priority
constraintStrength

requiredRoomId
requiredResourceIds
minimumRoomCapacity
```

Saved group examples

```
JHS 1 Combined Computing
Primary 4–6 Choir
Upper Primary French
All Prefects
Wednesday Clubs
SHS General Arts Elective Economics
```

Combination policy

Every requirement should specify:

```
combinationPolicy:
  | "never"
  | "suggest_only"
  | "allowed"
  | "preferred";
```

This prevents the recovery engine from proposing combinations for unsuitable subjects.

Phase 6 completion standard

- The system knows all required lessons.
- It knows lesson frequencies and lengths.
- It knows whether delivery is separate or combined.
- It knows whether future combination suggestions are permitted.
- It knows teacher, room, capacity, and resource requirements.

Phase 7 — Constraint, conflict and readiness engines

Goal

Create one validation system used by manual editing, generation, draft comparison, and recovery analysis.

New files

```
app/lib/scheduling/constraintEngine.ts
app/lib/scheduling/conflictEngine.ts
app/lib/scheduling/readinessEngine.ts
app/lib/scheduling/manualPlacementService.ts
```

Hard constraint functions

```
checkTeacherCollision()
checkParticipantGroupCollision()
checkStudentGroupCollision()
checkRoomCollision()
checkResourceCollision()

checkTeacherAvailability()
checkAllRequiredTeachersAvailable()

checkSharedBlockCollision()
checkTeachingSlot()
checkPeriodTemplateCompatibility()

checkRoomCapacity()
checkResourceCapacity()

checkConsecutiveSlotRequirement()
checkMaximumSubjectPeriodsPerDay()
checkTeacherWorkloadLimit()
```

Soft constraint functions

```
checkSubjectSpread()
checkTeacherPreferredSlot()
checkTeacherAvoidedSlot()
```

```
checkTeacherGaps()
checkPreferredFreeDay()
checkCoreSubjectTiming()
checkRepeatedSubjectSameDay()
checkDailyClassBalance()
checkRoomPreference()
```

`readinessEngine.ts`

Pre-generation checks:

```
checkPeriodTemplates()
checkRequirements()
checkTeachers()
checkTeacherAvailability()
checkWorkloadRules()
checkResources()
checkRoomCapacity()
checkCombinedGroupCompatibility()
checkTotalSlotCapacity()
checkLockedSessions()
```

New component

```
app/components/scheduling/SchedulingReadinessPanel.tsx
```

It shows:

- Ready.
- Warning.
- Blocking problem.
- Recommended fix.
- Link to the correct editor.

Conflict model extension

Extend conflict types beyond the existing teacher, class, student, room, and resource collision categories. The current model already supports those basic conflict references.

Add:

```
teacher_unavailable
teacher_overload
```

```
invalid_period_slot
shared_block_overlap
combined_group_incompatible
room_capacity_exceeded
required_resource_missing
required_teacher_missing
requirement_underallocated
requirement_overallocated
locked_session_blocking
```

Phase 7 completion standard

- Manual placements are checked immediately.
- Readiness is checked before generation.
- Conflict explanations identify the cause.
- Combined classes reserve every participating class.
- Multiple required teachers are validated together.
- The same rules are used everywhere.

Phase 8 — Standard timetable generation and draft scoring

Goal

Generate the best timetable possible without changing the school's intended structure.

New files

```
app/lib/scheduling/draftGenerator.ts
app/lib/scheduling/scoringEngine.ts
app/lib/scheduling/generationService.ts
app/lib/scheduling/draftRepository.ts
app/workers/timetableGenerator.worker.ts
app/hooks/useTimetableGenerator.ts
```

Standard generation sequence

```
1. Load scheduling inputs.
2. Build available period grid.
3. Reserve shared blocks.
```

```
4. Insert locked sessions.
5. Rank requirements by difficulty.
6. Generate valid candidate placements.
7. Reject hard violations.
8. Score remaining placements.
9. Place the best candidate.
10. Forward-check remaining requirements.
11. Backtrack where necessary.
12. Retry using controlled randomisation.
13. Improve completed solutions.
14. Save the strongest drafts.
```

Requirement difficulty ranking

Schedule difficult requirements first:

- Few available slots.
- Several combined classes.
- Multiple teachers.
- Required room.
- Limited resource.
- Double period.
- High-frequency subject.
- Teacher with restricted availability.
- Large group requiring high-capacity room.

Scoring areas

```
totalScore
teacherPreferenceScore
workloadBalanceScore
subjectDistributionScore
classBalanceScore
teacherGapScore
resourceUseScore
roomUseScore
combinedGroupScore
softPenaltyTotal
```

New components

```
app/components/scheduling/DraftGenerationPanel.tsx
app/components/scheduling/DraftComparison.tsx
app/components/scheduling/DraftScoreBreakdown.tsx
```

```
app/components/scheduling/SchedulingIssuePanel.tsx
app/components/scheduling/GenerationProgressPanel.tsx
```

Draft generation controls

- Number of drafts.
- Strictness.
- Iteration limit.
- Respect locked sessions.
- Prioritise teacher comfort.
- Prioritise subject spread.
- Minimise teacher gaps.
- Prefer room efficiency.
- Allow permitted soft exceptions.

Phase 8 completion standard

- The school can generate several drafts.
- Hard-invalid drafts are rejected.
- Drafts can be compared.
- Unscheduled requirements are clearly listed.
- No class combination is introduced unless already configured.

Phase 9 — Recovery analysis and class-combination recommendations

Goal

When standard generation cannot place all requirements, Eleeveon should not merely say “no solution.”

It should analyse the bottleneck and recommend reasonable structural changes.

Recovery hierarchy

The engine should try the least disruptive measures first:

```
1. Try more candidate placements.
2. Try additional backtracking.
3. Retry with a different requirement order.
4. Relax permitted soft preferences.
5. Move unlocked sessions.
6. Suggest a different suitable room.
```

```
7. Suggest another assigned teacher where permitted.
8. Suggest a partial class combination.
9. Suggest a full class combination.
10. Suggest moving a movable shared block.
11. Suggest extending the teaching day.
12. Report a genuinely impossible requirement.
```

New files

```
app/lib/scheduling/recoveryAnalysisEngine.ts
app/lib/scheduling/combinationSuggestionEngine.ts
app/lib/scheduling/suggestionSimulationEngine.ts
app/lib/scheduling/suggestionApplicationService.ts
```

`recoveryAnalysisEngine.ts`

Functions:

```
analyseGenerationFailure()
identifyBottlenecks()
rankBottlenecks()
findRecoverableIssues()
buildRecoveryStrategies()
estimateRecoveryImpact()
```

`combinationSuggestionEngine.ts`

Functions:

```
findCombinationCandidates()
scoreCombinationCompatibility()

validateSameSubject()
validateCurriculumCompatibility()
validateAcademicLevelCompatibility()
validateTeacherCompatibility()
validatePeriodCompatibility()
validateRoomCapacity()
validateResourceCapacity()
validateSchoolCombinationPolicy()
```

```
estimateCombinationBenefit()
buildCombinationSuggestion()
```

Reasonable combination criteria

A suggested combination should consider:

- Same subject.
- Same curriculum.
- Same or compatible academic level.
- Similar lesson objectives.
- Matching period frequency.
- Same teacher or permitted shared teacher.
- Compatible period templates.
- Matching free periods.
- Room capacity.
- Resource capacity.
- Total number of students.
- School-defined combination policy.
- Maximum permitted combined classes.
- Maximum permitted combined students.
- Whether the subject is suitable for group teaching.

Subjects more naturally suited to combination

Examples include:

- Computing demonstrations.
- Physical education.
- Music.
- Creative arts.
- Religious and moral education.
- Clubs.
- Assembly.
- Guest lectures.
- Certain electives.

The school should decide which subjects allow combinations.

Partial-combination suggestions

The engine should not always recommend combining every weekly period.

Example:

```
Original:
JHS 2A Computing — 3 periods
JHS 2B Computing — 3 periods

Suggested:
1 combined theory or demonstration period
2 separate practical periods for each class
```

Suggestion information shown to the administrator

```
Suggestion:
Combine JHS 1A and JHS 1B for Computing.

Reason:
The assigned teacher and ICT laboratory do not have enough separate
compatible periods.

Expected effect:
- Resolves 2 unallocated requirements.
- Saves 2 teacher periods.
- Saves 2 ICT laboratory periods.
- Combined students: 54.
- Room capacity: 60.
- Curriculum compatibility: High.
- Timetable impact: Moderate.
```

Administrator actions

```
Preview
Accept and regenerate
Modify selected classes
Apply to one period only
Save as permanent group
Apply to this draft only
Reject
Do not suggest again
```

Critical approval rule

The system must never silently combine classes.

The sequence must be:

```
Suggestion
    ↓
Administrator review
    ↓
Administrator approval
    ↓
Requirement transformation
    ↓
Regeneration
```

New components

```
app/components/scheduling/RecoveryAnalysisPanel.tsx
app/components/scheduling/CombinationSuggestionCard.tsx
app/components/scheduling/SuggestionImpactPreview.tsx
app/components/scheduling/SuggestionApprovalDialog.tsx
app/components/scheduling/RecoveryComparisonView.tsx
```

Phase 9 completion standard

- Failed generation is analysed.
- Bottlenecks are explained.
- Recommendations are ranked by benefit and disruption.
- Class combinations are academically and operationally validated.
- Suggestions require approval.
- Accepted suggestions can regenerate a revised draft.
- Rejected suggestions remain recorded to avoid repetition.

Phase 10 — Publishing, printing and portal projections

Goal

Complete the scheduling lifecycle after a draft has been accepted.

Major branch planner rebuild

```
app/branch-admin/BranchTimetable.tsx
```

Recommended view modes:

```
type TimetableViewMode =
  | "overview"
  | "planner"
  | "weekly"
  | "periods"
  | "shared_blocks"
  | "groups"
  | "requirements"
  | "availability"
  | "workload"
  | "readiness"
  | "drafts"
  | "recovery"
  | "issues"
  | "conflicts"
  | "analytics"
  | "publish";
```

Planner workflow

The branch administrator can:

1. Select academic period.
2. Select or create timetable.
3. Define period template.
4. Define shared blocks.
5. Create saved groups.
6. Define requirements.
7. Define availability.
8. Define workload.
9. Validate readiness.
10. Generate drafts.
11. Compare drafts.
12. Review unresolved requirements.
13. Review recovery suggestions.
14. Approve selected structural changes.
15. Regenerate.
16. Manually adjust.
17. Lock important sessions.
18. Regenerate unlocked sessions.
19. Commit final draft.
20. Publish.
21. Print.

New publishing files

```
app/lib/scheduling/publishService.ts
app/components/scheduling/PublishTimetablePanel.tsx
app/components/scheduling/TimetableVersionHistory.tsx
```

Publishing options

- Effective date.
- Version number.
- Publish to teachers.
- Publish to students.
- Publish to parents.
- Publish room timetable.
- Notify affected users.
- Archive previous timetable.
- Preserve prior version for audit.
- Unpublish or supersede.

Printing files

```
app/lib/scheduling/printTimetable.ts
app/components/scheduling/TimetablePrintView.tsx
app/components/scheduling/TimetablePrintSettings.tsx
```

Print formats

- Individual class.
- All classes.
- Individual teacher.
- All teachers.
- Branch master timetable.
- Room timetable.
- Resource timetable.
- A4 landscape.
- A3 landscape.
- Compact or spacious.
- Colour or monochrome.
- School logo and branch identity.
- Effective dates.
- Signature lines.
- Teacher names on or off.
- Room names on or off.

Portal files

Teacher

```
app/teacher/modules/MyTimetable.tsx
```

Shows:

- Assigned lessons.
- Combined classes.
- Co-teachers.
- Rooms.
- Free periods.
- Print view.

Student

```
app/student/modules/MyTimetable.tsx
```

Shows sessions applying to:

- Their active class.
- Their saved groups.
- Their programme or pathway.
- Their branch-wide activities.

Parent

```
app/parent/modules/MyChildTimetable.tsx
```

Shows:

- Child selector.
- Published timetable.
- Combined or group activities.
- Print view.

Room and resource administration

```
app/branch-admin/modules/RoomTimetable.tsx
app/branch-admin/modules/ResourceTimetable.tsx
```

Existing exam page

```
app/branch-admin/modules/ExamTimetable.tsx
```

It should reuse:

- Teacher conflict logic.
- Room conflict logic.
- Resource conflict logic.
- Grid and printing components.

Automatic exam generation should remain a later independent project.

Phase 10 completion standard

- One draft becomes the official timetable.
- Previous versions remain traceable.
- All role portals read the same published schedule.
- Timetables can be printed professionally.
- Combined sessions appear correctly everywhere.

---

4. Complete frontend file map

Database and sync updates

```
app/lib/db/db.ts
app/lib/db/db-version.ts
app/lib/db/db-migrations.ts
app/lib/sync/*
app/hooks/useDataRevision.ts
```

Scheduling domain files

```
app/lib/scheduling/types.ts
app/lib/scheduling/groupResolver.ts
app/lib/scheduling/timetableRepository.ts
app/lib/scheduling/draftRepository.ts
app/lib/scheduling/scheduleIdentity.ts
app/lib/scheduling/scheduleSelectors.ts
app/lib/scheduling/periodEngine.ts
app/lib/scheduling/availabilityEngine.ts
```

```
app/lib/scheduling/workloadEngine.ts
app/lib/scheduling/requirementBuilder.ts
app/lib/scheduling/readinessEngine.ts
app/lib/scheduling/constraintEngine.ts
app/lib/scheduling/conflictEngine.ts
app/lib/scheduling/manualPlacementService.ts
app/lib/scheduling/draftGenerator.ts
app/lib/scheduling/scoringEngine.ts
app/lib/scheduling/generationService.ts
app/lib/scheduling/recoveryAnalysisEngine.ts
app/lib/scheduling/combinationSuggestionEngine.ts
app/lib/scheduling/suggestionSimulationEngine.ts
app/lib/scheduling/suggestionApplicationService.ts
app/lib/scheduling/timetableProjection.ts
app/lib/scheduling/publishService.ts
app/lib/scheduling/printTimetable.ts
```

Hooks and worker

```
app/hooks/useSchedulingRevision.ts
app/hooks/useTimetableGenerator.ts
app/workers/timetableGenerator.worker.ts
```

Shared scheduling components

```
app/components/scheduling/WeeklyTimetableGrid.tsx
app/components/scheduling/TimetableCell.tsx
app/components/scheduling/PeriodHeader.tsx
app/components/scheduling/TimetableGridToolbar.tsx
app/components/scheduling/TimetableSessionEditor.tsx
app/components/scheduling/SessionParticipantSummary.tsx
app/components/scheduling/SessionLockControl.tsx

app/components/scheduling/PeriodTemplateEditor.tsx
app/components/scheduling/PeriodSlotEditor.tsx
app/components/scheduling/PeriodTemplateAssignmentEditor.tsx

app/components/scheduling/SharedBlockEditor.tsx
app/components/scheduling/SharedBlockScopeEditor.tsx

app/components/scheduling/ParticipantGroupEditor.tsx
app/components/scheduling/ScheduleGroupManager.tsx
app/components/scheduling/ScheduleGroupMemberEditor.tsx
```

```
app/components/scheduling/SubjectRequirementEditor.tsx
app/components/scheduling/RequirementTeacherEditor.tsx
app/components/scheduling/RequirementDistributionEditor.tsx
app/components/scheduling/ResourceRequirementEditor.tsx

app/components/scheduling/SchedulingReadinessPanel.tsx
app/components/scheduling/GenerationProgressPanel.tsx
app/components/scheduling/DraftGenerationPanel.tsx
app/components/scheduling/DraftComparison.tsx
app/components/scheduling/DraftScoreBreakdown.tsx
app/components/scheduling/SchedulingIssuePanel.tsx

app/components/scheduling/RecoveryAnalysisPanel.tsx
app/components/scheduling/CombinationSuggestionCard.tsx
app/components/scheduling/SuggestionImpactPreview.tsx
app/components/scheduling/SuggestionApprovalDialog.tsx
app/components/scheduling/RecoveryComparisonView.tsx

app/components/scheduling/PublishTimetablePanel.tsx
app/components/scheduling/TimetableVersionHistory.tsx
app/components/scheduling/TimetablePrintView.tsx
app/components/scheduling/TimetablePrintSettings.tsx
```

Existing branch files to rebuild or update

```
app/branch-admin/BranchTimetable.tsx
app/branch-admin/ClassTimetable.tsx
app/branch-admin/modules/TeacherTimetable.tsx
app/branch-admin/modules/TeacherAvailability.tsx
app/branch-admin/modules/TeacherWorkload.tsx
app/branch-admin/modules/ExamTimetable.tsx
```

New branch pages

```
app/branch-admin/modules/RoomTimetable.tsx
app/branch-admin/modules/ResourceTimetable.tsx
```

Portal files

```
app/teacher/modules/MyTimetable.tsx
app/student/modules/MyTimetable.tsx
app/parent/modules/MyChildTimetable.tsx
```

5. Complete backend file map

Prisma

```
backend/prisma/schema.prisma
backend/prisma/migrations/<timestamp>_advanced_timetable_engine/
```

Scheduling module

```
backend/src/scheduling/scheduling.module.ts
backend/src/scheduling/scheduling.controller.ts
backend/src/scheduling/scheduling.service.ts
```

DTOs

```
backend/src/scheduling/dto/create-period-template.dto.ts
backend/src/scheduling/dto/create-period-slot.dto.ts
backend/src/scheduling/dto/create-shared-block.dto.ts

backend/src/scheduling/dto/create-schedule-group.dto.ts
backend/src/scheduling/dto/update-schedule-group.dto.ts

backend/src/scheduling/dto/create-teacher-availability.dto.ts
backend/src/scheduling/dto/create-workload-rule.dto.ts

backend/src/scheduling/dto/create-subject-requirement.dto.ts
backend/src/scheduling/dto/create-resource-requirement.dto.ts
backend/src/scheduling/dto/create-constraint-rule.dto.ts

backend/src/scheduling/dto/create-generation-run.dto.ts
backend/src/scheduling/dto/save-draft.dto.ts
backend/src/scheduling/dto/save-generation-issue.dto.ts

backend/src/scheduling/dto/save-generation-suggestion.dto.ts
backend/src/scheduling/dto/apply-generation-suggestion.dto.ts

backend/src/scheduling/dto/commit-draft.dto.ts
backend/src/scheduling/dto/publish-timetable.dto.ts
```

Sync updates

Update backend mappings for every new table under:

```
backend/src/sync/
```

The local Web Worker will perform generation initially. The backend will primarily persist, sync, audit, and publish the resulting data.

---

6. Exact implementation order

Build Set 1 — Database foundation

```
1. Review and update db.ts scheduling types.
2. Add new scheduling interfaces.
3. Add Dexie table declarations.
4. Add Dexie store indexes.
5. Increase db-version.ts.
6. Add db migration.
7. Update frontend sync registry.
8. Update Prisma schema.
9. Add Prisma migration.
10. Update backend sync registry.
```

Build Set 2 — Scheduling domain

```
11. types.ts
12. scheduleIdentity.ts
13. groupResolver.ts
14. scheduleSelectors.ts
15. timetableRepository.ts
16. useSchedulingRevision.ts
```

Build Set 3 — Period structures

```
17. periodEngine.ts
18. PeriodTemplateEditor.tsx
19. PeriodSlotEditor.tsx
20. PeriodTemplateAssignmentEditor.tsx
```

21. SharedBlockEditor.tsx
22. SharedBlockScopeEditor.tsx

Build Set 4 — Timetable grid

23. WeeklyTimetableGrid.tsx
24. TimetableCell.tsx
25. PeriodHeader.tsx
26. TimetableGridToolbar.tsx
27. TimetableSessionEditor.tsx
28. SessionParticipantSummary.tsx
29. SessionLockControl.tsx
30. Initial ClassTimetable.tsx rebuild

Build Set 5 — Teacher rules

31. availabilityEngine.ts
32. workloadEngine.ts
33. TeacherAvailability.tsx
34. TeacherWorkload.tsx
35. TeacherTimetable.tsx

Build Set 6 — Requirements and participant groups

36. requirementBuilder.ts
37. ParticipantGroupEditor.tsx
38. ScheduleGroupManager.tsx
39. ScheduleGroupMemberEditor.tsx
40. SubjectRequirementEditor.tsx
41. RequirementTeacherEditor.tsx
42. RequirementDistributionEditor.tsx
43. ResourceRequirementEditor.tsx

Build Set 7 — Validation

44. readinessEngine.ts
45. constraintEngine.ts
46. conflictEngine.ts
47. manualPlacementService.ts
48. SchedulingReadinessPanel.tsx
49. Integrate validation into BranchTimetable.

50. Integrate validation into ClassTimetable.
51. Integrate validation into TeacherTimetable.

Build Set 8 — Standard generation

52. scoringEngine.ts
53. draftGenerator.ts
54. draftRepository.ts
55. generationService.ts
56. timetableGenerator.worker.ts
57. useTimetableGenerator.ts
58. GenerationProgressPanel.tsx
59. DraftGenerationPanel.tsx
60. DraftComparison.tsx
61. DraftScoreBreakdown.tsx
62. SchedulingIssuePanel.tsx

Build Set 9 — Recovery recommendations

63. recoveryAnalysisEngine.ts
64. combinationSuggestionEngine.ts
65. suggestionSimulationEngine.ts
66. suggestionApplicationService.ts
67. RecoveryAnalysisPanel.tsx
68. CombinationSuggestionCard.tsx
69. SuggestionImpactPreview.tsx
70. SuggestionApprovalDialog.tsx
71. RecoveryComparisonView.tsx
72. Regeneration after approved suggestions.

Build Set 10 — Planner completion and publishing

73. Complete BranchTimetable.tsx rebuild.
74. Complete ClassTimetable.tsx rebuild.
75. timetableProjection.ts
76. publishService.ts
77. PublishTimetablePanel.tsx
78. TimetableVersionHistory.tsx
79. printTimetable.ts
80. TimetablePrintView.tsx
81. TimetablePrintSettings.tsx

Build Set 11 — Portal delivery

```
82. teacher/MyTimetable.tsx
83. student/MyTimetable.tsx
84. parent/MyChildTimetable.tsx
85. RoomTimetable.tsx
86. ResourceTimetable.tsx
87. ExamTimetable shared validation updates
```

---

7. Features deliberately postponed

The first advanced version should not include:

- AI language-model timetable generation.
- Cloud-only optimisation.
- Automatic exam timetable generation.
- Cross-branch scheduling.
- Automatic substitute-teacher scheduling.
- Rotating Week A and Week B schedules.
- Two-week or monthly cycles.
- Transport timetable optimisation.
- Fully individualized student elective schedules.
- Automatic changes to academic delivery without approval.

The architecture should allow these later without requiring a complete redesign.

---

8. Final definition of completion

The timetable engine is complete when a branch administrator can:

1. Define school operating days.
2. Define periods and times.
3. Define different templates for different levels.
4. Reserve assembly, breaks, lunch, worship, sports, and closing.
5. Create custom participant groups.
6. Configure single-class lessons.
7. Configure combined-class lessons.
8. Configure partial combinations.
9. Assign multiple teachers.
10. Assign rooms and resources.
11. Define teacher availability.

12. Define teacher workload limits.
13. Define required subject periods.
14. Define whether a subject may be combined.
15. Validate scheduling readiness.
16. Manually create and move sessions safely.
17. Generate several standard timetable drafts.
18. Compare draft scores.
19. See every unresolved requirement.
20. Understand why each requirement failed.
21. Receive reasonable recovery recommendations.
22. Preview suggested class combinations.
23. Approve or reject every proposed combination.
24. Regenerate using approved suggestions.
25. Lock important sessions.
26. Regenerate only unlocked sessions.
27. Commit the selected draft.
28. Publish the official timetable.
29. Preserve previous versions.
30. Print class, teacher, room, resource, and branch timetables.
31. Let teachers see their own timetable.
32. Let students see their class and group timetable.
33. Let parents see each child's timetable.
34. Prevent class, teacher, student, room, and resource collisions.
35. Avoid silently changing the school's academic structure.

The most important principle is:

> Eleeveon should first respect the school's intended timetable structure. When that structure cannot fit, it should explain the bottleneck and intelligently recommend the least disruptive reasonable solution—such as combining compatible classes—but only the school can approve and apply that decision.